New Relic APM — Day 1 Study Guide: Transactions, Traces and Services

Oct 26, 2026 · @Vivek Arora

Day 1 at a glance

Day 1 is about **finding the cause of a production slowdown or failure fast**, using three layers of New Relic APM data: the transactions of a single service, the distributed traces that cross services, and the correlated context around them — service maps, logs and change records. Every concept is practised on **easyTravel**, a multi-tier Java travel-booking application, running on a Linux VM and instrumented with the **New Relic Java agent**.

Who it is for

SREs, production support and DevOps engineers, and developers who already know the basics of New Relic (accounts, the APM services list, simple NRQL) and now need to use it under pressure during incidents.

Learning objectives

By the end of Day 1 you can:

1. Read the APM Summary during an incident and decide within minutes whether the problem is web traffic, background work, the database, an external call or the code itself.
2. Open a transaction trace and point to the exact slow segment, including N+1 query patterns and repeated external calls.
3. Explain how W3C Trace Context carries a request across services, and recognise the usual reasons a trace breaks.
4. Follow one failed request end to end in the trace waterfall and name the failing hop.
5. Choose between head-based sampling and Infinite Tracing (tail-based) and say what each one can miss.
6. Query `Span`, `Transaction` and `Log` data with NRQL to confirm what the UI shows.
7. Use the service map to judge upstream and downstream impact (blast radius) during triage.
8. Jump from a trace or an error straight to its log lines, and correlate a regression with a deployment marker.

Agenda

Time	Session	Lab
09:30 – 09:50	Day 1 kick-off; the easyTravel lab system	Lab 1.0 — Instrument easyTravel (pre-work or start of day)
09:50 – 11:50	**Module 1** — Transaction analysis for production troubleshooting	Lab 1.1 — Root-cause a slow checkout (45 min)
11:50 – 12:05	Break	
12:05 – 13:15	**Module 2** — Distributed tracing in microservices (theory)	
13:15 – 14:00	Lunch	
14:00 – 14:50	Module 2 (continued)	Lab 1.2 — Follow a failed request end to end (50 min)
14:50 – 15:05	Break	
15:05 – 17:00	**Module 3** — Service maps, dependencies and correlated telemetry	Lab 1.3 — Correlate a post-deployment latency spike (50 min)
17:00 – 17:30	Incident runbook review, knowledge check, Q&A	

How to use this guide with the lab guide

This study guide explains the concepts and the reasoning. Every hands-on step — commands, configuration, what to click and what you should see — lives in the separate **New Relic APM — Day 1 Lab Guide**. Each module ends with a *"Do Lab 1.x"* pointer, and each lab names the module it practises, so the two always stay in step.

Screens in the lab guide are drawn illustrations of the New Relic UI, labelled **Expected view**. The New Relic UI changes often: menu names and layouts on your account may differ slightly, but the data and the reasoning are the same.

What you need

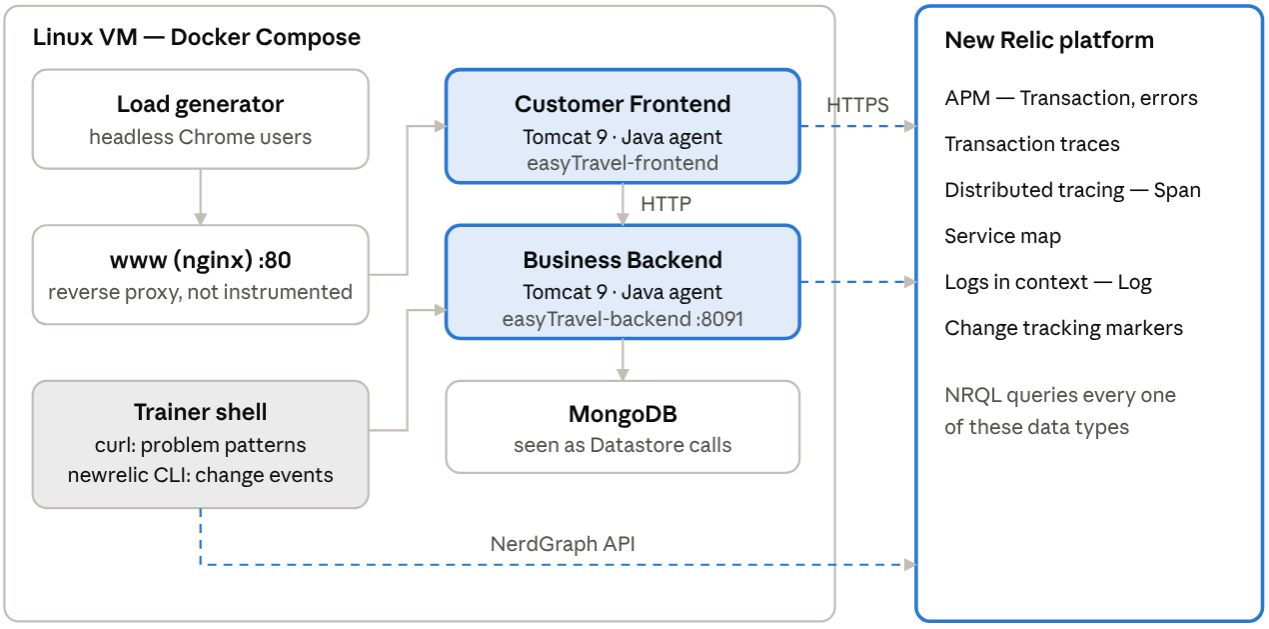
- A New Relic account (the free tier is enough for Labs 1.0–1.3) and an **ingest license key**.

- A Linux VM: Ubuntu 22.04 or 24.04, 4 vCPU, 8 GB RAM, Docker Engine with the Compose plugin, outbound HTTPS to New Relic.
- A browser to open New Relic and the easyTravel web shop.
- Day 1 assumes no prior Java knowledge; you never change easyTravel's code.

The lab system: easyTravel on Linux

easyTravel is a travel-booking demo application originally published by Dynatrace for performance training. It is no longer actively developed, but its official Docker images still give us exactly what Day 1 needs: a realistic multi-tier Java system with a database, real user-like traffic, and a library of switchable **problem patterns** (slow queries, N+1 database access, errors, CPU load) that we can turn on and off on demand.

Two instrumented Java tiers send traces, events and logs to New Relic



The Day 1 lab system · easyTravel on Docker with two New Relic Java agents

Components

Component	Container	What it does	Instrumented as
Load generator	`headless-customer-loadgen`	Headless Chrome "customers" who search, log in and book trips around the clock	— (traffic source)
www	`www` (nginx)	Reverse proxy in front of the web shop on port 80	Not instrumented — it passes trace headers through

Component	Container	What it does	Instrumented as
Customer Frontend	`frontend` (Tomcat 9, Java 11)	The web shop (JSF pages): search, login, booking, payment	APM service **easyTravel-frontend**
Business Backend	`backend` (Tomcat 9, Java 11)	Web services the frontend calls: journeys, bookings, authentication, plus the `ConfigurationService` that switches problem patterns	APM service **easyTravel-backend**
MongoDB	`mongodb`	Stores journeys, bookings, customers	Appears as **Datastore** calls inside the backend's transactions

The frontend and the backend are separate services that talk over HTTP. That makes easyTravel a small but genuine distributed system: one click in the browser becomes a frontend transaction, an outbound **external** call, a backend transaction and several **datastore** calls — all tied together by one trace ID.

What the Java agent sends

The New Relic Java agent attaches to each Tomcat JVM with `-javaagent` and, with no code change:

- names and times every web request as a **transaction** (and background work as a non-web transaction);
- records **metrics** for each transaction, segment, datastore operation and external host;
- captures **transaction traces** for slow requests, with their call tree and SQL/MongoDB statements;
- creates **spans** for distributed tracing and propagates the **W3C Trace Context** headers on outbound HTTP calls (distributed tracing is on by default from Java agent 7.4.0);
- reports **errors** with stack traces;
- forwards **application logs**, decorated with the trace and span IDs (logs in context).

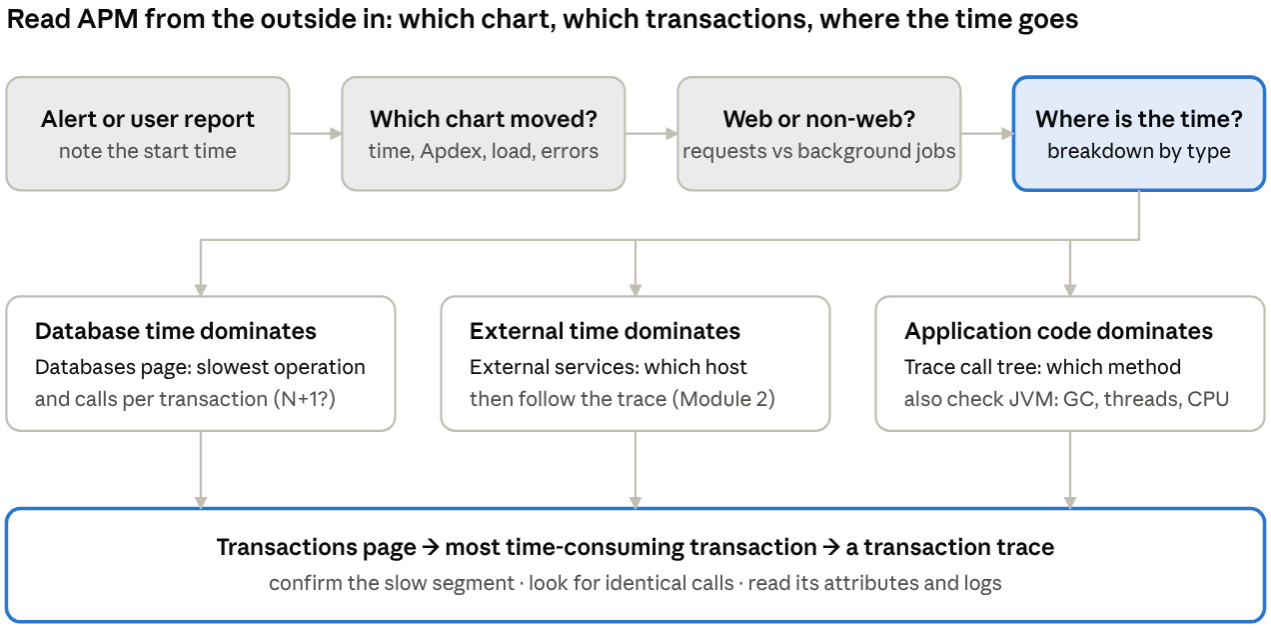
Problem patterns we use on Day 1

Pattern	What it simulates	Used in
`DBSpamming`	Unnecessary repeated database access when a booking is stored — an N+1 pattern	Lab 1.1
(a stopped MongoDB)	A dependency outage that makes downstream calls fail	Lab 1.2
`DBSpammingAuthWithAppDeployment`	Simulates a redeployment, then starts database spamming for logged-in users about a minute later	Lab 1.3

Patterns are switched on and off with the backend's `ConfigurationService` (two `curl` calls, shown in the lab guide). Turn each one off after its lab so the next lab starts from a clean baseline.

Module 1 — Transaction analysis for production troubleshooting

A **transaction** is one unit of work the agent times from start to finish — a web request, a message consumed from a queue, a scheduled job. Everything APM shows you is built from transactions and the **segments** inside them. During an incident the skill is to move from the broad picture to the one slow segment in as few clicks as possible.



Reading APM during an incident · four questions, three branches, one trace

1.1 Web vs non-web (background) transactions

	Web transactions	Non-web transactions
Started by	An inbound HTTP request (servlet, JSF page, REST endpoint)	Anything else: scheduled jobs, message consumers, batch work, threads the app starts itself
Named like	`WebTransaction/...` (e.g. `/orange-booking-finish.jsf`)	`OtherTransaction/...` (e.g. `OtherTransaction/Job/...`)
Apdex	Yes — users are waiting	No Apdex: nobody is waiting on a page
Summary chart	**Web transactions time**	Switch the chart to **Non-web transactions time**

Incidents often hide in the transaction type you are not looking at. A nightly cleanup job (easyTravel's `DatabaseCleanup` is one) can hold database locks or saturate CPU and make web requests slow, while the web chart only shows the symptom. **Always check both views** before deciding where the problem is.

1.2 Reading the APM Summary during an incident

The Summary page of a service answers four questions in order:

1. **Which chart moved, and when?** Response time, Apdex, throughput or error rate. Pin the start time — every later question uses it.
2. **Is it all transactions or a few?** The *Transactions* list sorted by **Most time consuming** shows which ones carry the load. One transaction jumping from 300 ms to 3 s is a different incident from every transaction slowing by 20%.
3. **Where is the time going?** The response-time chart is stacked by segment type — application code (Java), each database, external calls. The band that grew is your first lead.
4. **Did anything change?** Deployment markers (Module 3) appear as vertical lines across the charts.

Throughput is the context for everything else: response time rising while throughput is flat points to a dependency or a code change; response time rising **with** throughput points to capacity. A drop in throughput together with a rise in errors usually means requests fail fast.

1.3 Transaction traces: finding the bottleneck

Metrics tell you *which* transaction and *which category* is slow; a **transaction trace** shows one real request in detail. The Java agent captures a trace when a transaction is slower than `transaction_tracer.transaction_threshold` — by default `apdex_f`, which is 4 × the Apdex T (with the default T of 0.5 s, anything over 2 s).

In the trace you see:

- **Trace details** — the call tree in timeline, latency or waterfall form. Selecting a segment opens its **Performance**, **Attributes**, **Details** and **Logs** tabs.
- **Slow spans** — a leaf or exit segment whose exclusive time is at least 25% of the trace is highlighted, and *Focus on slow spans* filters to just those.
- **Grouped calls** — many quick calls (each under about 7.5 ms) are folded into one *fast calls* group, and **four or more consecutive identical calls** are collapsed into an *Identical Calls* group. Expand them: this is where N+1 patterns show up.
- **Database statements** — SQL (obfuscated by default) or MongoDB operations, with explain plans for SQL slower than `explain_threshold` (0.5 s).

The N+1 pattern. Code loads a list (1 query), then loops over it and queries again for each item (N queries). Each query is fast, so no single statement looks slow — but 240 calls × 11 ms is 2.6 s. Signs:

- the trace shows an *Identical Calls* group of the same datastore operation;
- the transaction's breakdown table shows a datastore segment with a high **average calls per transaction**;
- total database time is high while the average time per call is low.

The fix is in the code (batch the reads, join, cache), but proving it with the trace is what gets it prioritised.

1.4 Segments, call trees and breakdown tables

A **segment** is a timed piece of a transaction: a method, a datastore call, an external call. Two numbers matter for each one:

Measure	Meaning	Use it to
Total (inclusive) time	The segment plus everything it called	Follow the expensive branch down the tree
Exclusive (self) time	Time spent in the segment itself, not in its children	Find the segment that actually does the slow work

The **breakdown table** on a transaction's page aggregates every request of that transaction: for each segment, **% of time**, **average calls per transaction**, **average time** and **average total time**. It is the quickest way to tell "one slow call" (high avg time, ~1 call) from "too many calls" (low avg time, many calls).

1.5 External services and database calls

Page	What it shows	Questions to ask
Databases	Each datastore operation (e.g. `MongoDB Journey find`, `MySQL orders select`): time consumption, query time, throughput, and the transactions that call it	Did one operation get slower, or did it get called more? Which transactions are responsible?
External services	Each outbound host: response time, throughput, and which transactions call it	Is the slowness in our code or in the service we call? Does the downstream service show the same latency in its own APM?

A failing dependency hurts in three ways: **latency** (callers wait), **errors** (exceptions propagate), and **throughput** (threads and connection pools fill up, so even unrelated requests queue). When an external host's response time rises, open that service's own APM — if its server-side time is flat, the time is being lost on the network, in a proxy or in a connection pool between the two.

1.6 Key transactions and naming problems

Key transactions mark the few transactions that matter most to the business — in easyTravel, login, journey search and booking. A key transaction can have **its own Apdex T** (web transactions only), its own NRQL alert conditions and service levels, and can be added to dashboards and workloads. Create one from *APM & Services → Transactions → select the transaction → Make key transaction*. Note that its Apdex T overrides the app's for that transaction.

Naming problems hide incidents. The value of transaction data depends on good names:

Problem	What you see	Why it hurts	Fix
Too coarse	Everything under one name (`/*` , a single servlet or controller)	A slow checkout is averaged away by fast page loads	Name by route or action: agent API `NewRelic.setTransactionName(...)` or framework naming
Too fine (metric grouping issue, MGI)	IDs in names (`/booking/12345` , `/user/ravi@...`)	Thousands of names; the agent and New Relic start grouping them, and real problems vanish into the noise	Remove IDs from names; normalise with naming rules
Wrong type	Background work reported as web, or web work under `OtherTransaction`	Wrong Apdex, wrong charts	Use the right API or annotation (`@Trace(dispatcher = true)` for custom background work)
Unknown or `Unknown`	Requests the agent could not name	Hard to search and alert on	Check framework support; add custom instrumentation

Check names on day one of instrumenting a service — not in the middle of an incident.

1.7 NRQL to confirm what the UI shows

```
-- Slowest web transactions right now (95th percentile), easyTravel frontend
SELECT percentile(duration, 95) AS 'p95 (s)', count(*) FROM Transaction
WHERE appName = 'easyTravel-frontend' AND transactionType = 'Web'
FACET name SINCE 30 minutes ago

-- Where the time goes inside the booking call, minute by minute
-- (request.uri separates operations even when they share one transaction
name)
SELECT average(databaseDuration), average(externalDuration),
average(duration)
FROM Transaction WHERE appName = 'easyTravel-backend' AND request.uri LIKE
'%Booking%'
TIMESERIES 1 minute SINCE 30 minutes ago

-- How many database calls each request makes (the N+1 signal)
SELECT average(databaseCallCount), max(databaseCallCount) FROM Transaction
WHERE appName = 'easyTravel-backend' FACET name, request.uri SINCE 30 minutes
ago

-- Background (non-web) work
SELECT count(*), average(duration) FROM Transaction
WHERE transactionType = 'Other' FACET appName, name SINCE 1 hour ago
```

> **Do Lab 1.1 — Root-cause a slow checkout** in the Day 1 Lab Guide (45 min): turn on the `DBSpamming` pattern, see the booking transaction slow down on the Summary, find it in the Transactions list and the breakdown table, open a trace, expand the *Identical Calls* group, and confirm the N+1 pattern with NRQL.

Module 2 — Distributed tracing in microservices

A transaction trace (Module 1) shows one service's view of a request. As soon as that service calls another one, the trail ends at an **External** segment: you know the call was slow, but not why. **Distributed tracing** stitches the work of every service that touched the request into one **trace**, so the whole journey can be read in one place.

2.1 Why it matters in an incident

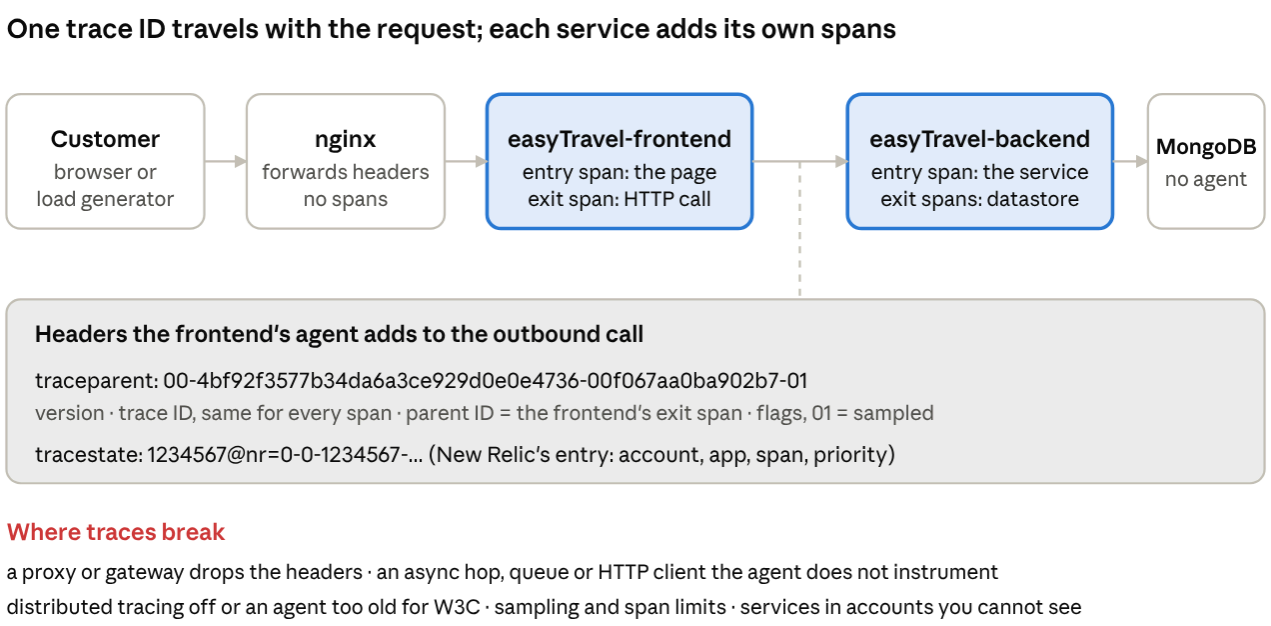
In a microservice system the service that alerts is rarely the service that broke. A checkout page is slow because the booking service is slow because a database lock is held by a reporting job. Without a trace you look at three dashboards, owned by three

teams, and compare timestamps by hand. With a trace you open one request and read where the time went and which hop returned the error.

Key vocabulary:

Term	Meaning
Trace	Everything that happened for one request, across all services; identified by one `trace.id`
Span	One timed operation in the trace: a service's transaction, a method, a datastore query, an external call
Entry span	The first span in a process — where a service receives the request
Exit span	A span that calls out of the process (an HTTP call, a datastore query); its child entry span belongs to the next service
Root span	The first span of the whole trace; its service is the trace's *root entity*

2.2 W3C Trace Context and propagation



Trace context propagation through easyTravel · W3C traceparent and tracestate

When an instrumented service makes an outbound call, its agent adds trace headers; the next agent reads them and continues the same trace.

- **`traceparent`** — the W3C standard header: version, the 32-hex **trace ID**, the 16-hex **parent span ID** (the caller's exit span) and the **sampled flag**.
- **`tracestate`** — vendor-specific data. New Relic's entry starts with the trusted account key and `@nr=`, and carries the account, app, span, transaction and sampling priority.
- **`newrelic`** — New Relic's older proprietary header, still sent for backward compatibility unless `distributed_tracing.exclude_newrelic_header: true`. When both are present, current agents use `traceparent`.

Because W3C Trace Context is a standard, a trace can also pass through services instrumented with OpenTelemetry or other vendors' agents, as long as each hop forwards the headers.

Common causes of broken traces

Cause	Symptom	Fix
A proxy, gateway or service mesh drops or rewrites headers	The downstream service starts its own traces; the upstream trace ends at an External span	Configure it to pass `traceparent`, `tracestate` and `newrelic`
Async work, thread pools, message queues	Spans after the hand-off are missing or start a new trace	Use supported libraries, or the agent's async/token and distributed-tracing APIs
An HTTP client or framework the agent does not instrument	No exit span and no headers on that call	Custom instrumentation, or a supported client
Distributed tracing disabled, or an agent too old for W3C	That service never joins traces	Enable it (`distributed_tracing.enabled: true`) and upgrade the agent
Sampling, span limits	Some requests have no trace, or a trace has gaps	Expected with head-based sampling; see 2.4
Services in an account you cannot access	Spans shown as obfuscated	Ask for access to that account
Spans arriving more than 24 hours late, clock skew	Missing or oddly placed spans	Fix clocks; avoid very long-running traces

In Lab 1.2 you break easyTravel's trace on purpose by disabling distributed tracing on the backend, and see exactly what a broken trace looks like.

2.3 Reading a trace: waterfall, attributes and error spans

The **distributed tracing** UI (APM → a service → *Distributed tracing*, or the global view) lists **trace groups** — traces with the same root entity and root span name — with duration, span count, number of services and errors. Filter by `errors`, by duration, or by any span attribute.

Opening a trace shows the **waterfall**: one row per span, indented under its parent, coloured by service, with a bar for its start and duration. Read it like this:

1. **Top to bottom** for the path of the request: entry span → work → exit span → next service's entry span.
2. **Longest bar that has no long children** — that is where the time is really spent (the same idea as exclusive time).
3. **Gaps** between a parent and its first child — time in code that is not instrumented, a queue, or the network.
4. **Red (error) spans** — the deepest red span is usually the origin of the failure; red spans above it are just the error propagating up.

Selecting a span shows its **attributes**. The most useful ones during an incident:

Attribute	Example	Why it helps
`trace.id`, `id`, `parentId`	`4bf92f35…`	Search, and join to logs (Module 3)
`entity.name` / `appName`	`easyTravel-backend`	Which service
`name`	`Datastore/statement/MongoDB/Journey/find`	What the span did
`duration.ms`	`2640`	How long
`span.kind`	`server`, `client`, `internal`	Entry, exit or in-process
`category`	`http`, `datastore`, `generic`	Kind of work
`http.url`, `http.statusCode`	`…/BookingService/storeBooking`, `500`	External calls

Attribute	Example	Why it helps
`db.statement` / database attributes	`find Journey`	Datastore calls
`error.class`, `error.message`	`com.mongodb.MongoTimeoutException`	Why it failed
`nr.entryPoint`	`true`	Marks a service's entry span

2.4 Head-based vs tail-based (Infinite Tracing) sampling

No system keeps every span of every request for free, so traces are **sampled**. The question is *when* the decision is made.

	Head-based (standard distributed tracing)	Tail-based (Infinite Tracing)
Decision made	At the **start** of the trace, by the first agent; passed downstream in the sampled flag	At the **end**, after all spans arrive at a **trace observer**
How it chooses	Adaptive sampler: most agents target **10 sampled transactions per minute** per agent (configurable 1–120); upstream-sampled traces are always kept. Newer agents also offer ratio, always-on and always-off samplers	Keeps traces that are **slow** (duration outliers), have **at least one error span**, plus a **random 1%** by default
What you send	Only sampled spans	**100% of spans** from the agents to the trace observer
What you miss	Rare errors and slow outliers may simply not be sampled; you see a representative picture	Very little of what matters; normal traces are mostly discarded
Cost and setup	Default; no setup	A trace observer per region; more egress from your network; check availability for your account

	Head-based (standard distributed tracing)	Tail-based (Infinite Tracing)
Configure (Java)	`distributed_tracing.enabled: true` (default)	plus `infinite_tracing.trace_observer.host: "<your trace observer host>"`

Practical rule: head-based sampling is right for understanding normal behaviour and most incidents; if your incidents are rare, intermittent errors that the sampler keeps missing, Infinite Tracing is the tool. Metrics, transaction events and error events are **not** sampled the same way — so a spike in the error rate is always visible even when no trace of it was kept.

2.5 Querying Span data with NRQL

Every span is stored as a `Span` event, so NRQL can answer questions the trace UI cannot — across thousands of traces at once.

```
-- Slowest downstream (exit) calls made by the frontend, by span name
SELECT count(*), average(duration.ms), percentile(duration.ms, 95)
FROM Span WHERE appName = 'easyTravel-frontend' AND span.kind = 'client'
FACET name SINCE 30 minutes ago

-- Where errors originate: error spans by service and span name
SELECT count(*) FROM Span WHERE error.class IS NOT NULL
FACET appName, name, error.class SINCE 30 minutes ago

-- Find specific slow traces to open in the UI
SELECT trace.id, appName, name, duration.ms FROM Span
WHERE nr.entryPoint IS true AND duration.ms > 2000
SINCE 30 minutes ago LIMIT 20

-- Every span of one trace, in order
SELECT timestamp, appName, name, duration.ms, error.class FROM Span
WHERE trace.id = '4bf92f3577b34da6a3ce929d0e0e4736' ORDER BY timestamp LIMIT
100

-- Datastore operations in the backend, slowest first
SELECT count(*), average(duration.ms) FROM Span
WHERE appName = 'easyTravel-backend' AND category = 'datastore'
FACET name SINCE 30 minutes ago
```

Span data is sampled (2.4), so use it to find *which* hop and *which* pattern; use `Transaction` and metric data for exact counts and rates.

> **Do Lab 1.2 — Follow a failed request end to end** in the Day 1 Lab Guide (50 min): read a healthy easyTravel trace, stop MongoDB to make bookings fail, find a failed trace, walk the waterfall to the failing datastore span, confirm with Span NRQL — then disable distributed tracing on the backend to see a broken trace and fix it.

Module 3 — Service maps, dependencies and correlated telemetry

Modules 1 and 2 answered *where* a request spends its time. Module 3 answers the questions an incident commander asks next: **who else is affected, what depends on what, and what changed?** The answer comes from putting several kinds of telemetry side by side — maps, logs and change records — on the same timeline.

3.1 Service maps and dependencies during triage

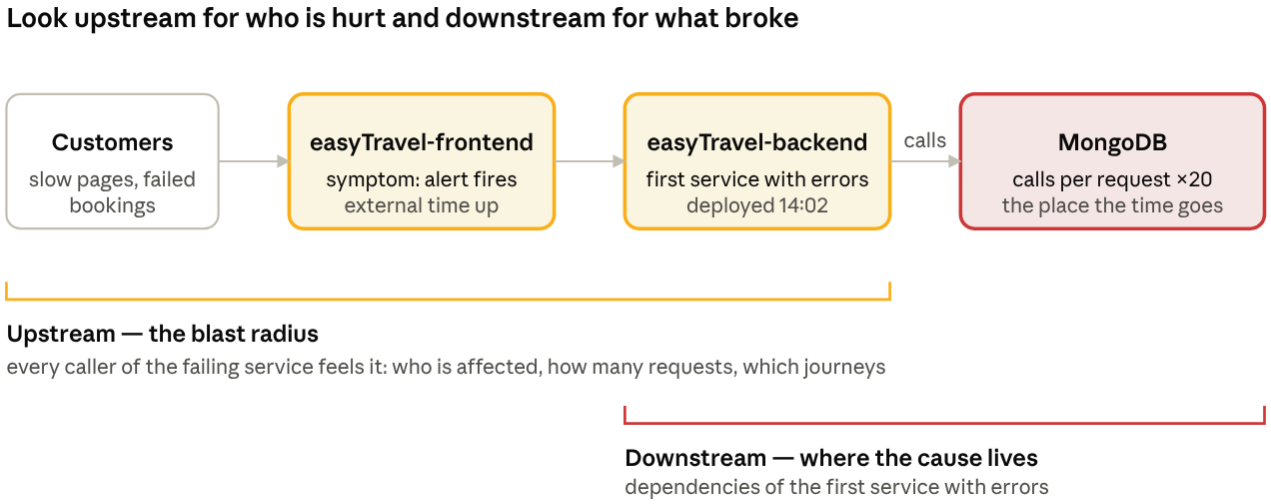
A **service map** draws each instrumented service, database and external host as a node and each call between them as a line. New Relic builds it from the same span and metric data you used in Module 2, so it is always current. On a service's map you can see, for the selected time window:

- **Nodes**: services (with health colour from their alert status), databases, external hosts, browser and mobile apps.
- **Connections**: throughput, response time and error rate of the calls between two nodes.
- **Direction**: who calls whom — the basis of upstream and downstream analysis.

Open it from *APM & Services → a service → Service map*, or from a distributed trace (the map of just that trace). The service's **Dependencies** / related-entities view shows the same relationships as a list, which is easier to scan when a service has many neighbours.

Use the map early in an incident to answer three questions in a minute: *Is the alerting service the only one in trouble? Which of its dependencies is also degraded? Which services call it?*

3.2 Upstream vs downstream: blast radius



Upstream vs downstream impact · the easyTravel example used in Lab 1.3

Direction	Question	Where to look
Downstream (what the service calls)	*What could be causing this?*	Databases, external services and called services on the map; the deepest error span in traces
Upstream (what calls the service)	*Who is affected, and how badly?*	Callers on the map; their error rates and Apdex; the trace groups that include the failing span

Work it in two passes:

1. **Go downstream to the cause.** Start at the alerting service and move along its dependencies to the **first service whose own errors or latency rose at the same time**. That service, or one of its datastores, is your root-cause candidate. A service that is slow only because it waits on its dependency is a *symptom*.
2. **Go upstream to the impact.** From the root-cause candidate, list every caller and the user journeys behind them. That list is the **blast radius**: it decides severity, who to notify, and what to check after the fix.

```
-- Blast radius: which services and entry points are seeing errors right now
SELECT count(*) AS 'error spans' FROM Span
WHERE error.class IS NOT NULL FACET appName, name SINCE 15 minutes ago

-- Which root transactions (user journeys) include the failing backend call
SELECT uniqueCount(trace.id) FROM Span
WHERE appName = 'easyTravel-backend' AND error.class IS NOT NULL
FACET name SINCE 15 minutes ago
```

3.3 Logs in context: from a trace or an error to its log lines

With **logs in context**, the Java agent forwards the application's log lines to New Relic and adds `trace.id`, `span.id`, `entity.guid`, `entity.name` and `hostname` to each one. The supported Java logging frameworks are Log4j 1 (1.2.17+), Log4j 2 (2.6+), Logback (1.1+), java.util.logging and JBoss Logging. Forwarding is on by default from agent 7.7.0.

That makes three jumps possible:

From	Click	You get
A span in a trace	the span's **Logs** tab	Only the lines written during that span's request
An error (Errors inbox, error trace)	**See logs** / the Logs tab	The lines around that exception, for that request
A log line in Logs	its `trace.id` link	The full distributed trace that produced it

```
-- Every log line written for one request
SELECT timestamp, entity.name, level, message FROM Log
WHERE trace.id = '4bf92f3577b34da6a3ce929d0e0e4736' ORDER BY timestamp

-- Error-level logs from the backend, grouped by message, since the alert
SELECT count(*) FROM Log WHERE entity.name = 'easyTravel-backend' AND level =
'ERROR'
FACET message SINCE 30 minutes ago
```

```
# newrelic.yml (common section) — logs in context, Java agent defaults
written out
application_logging:
  enabled: true
  forwarding:
    enabled: true
    max_samples_stored: 10000   # log lines per minute the agent will forward
  local_decorating:
    enabled: false              # true only if another log shipper sends the
file
  metrics:
    enabled: true               # log volume by level, even if forwarding is
off
```

Log forwarding sends data to New Relic like any other telemetry: keep `max_samples_stored` and log levels sensible, and never log secrets or personal data — obfuscate them with drop/obfuscation rules if they slip through.

3.4 Change tracking and deployment markers

Most production incidents start with a change. **Change tracking** records deployments and other changes (feature flags, configuration, infrastructure) against an entity; each one appears as a **marker** — a vertical line — on that entity's APM charts, with its version, commit, user, description and a link to the changelog.

The change tracking views add an **impact summary** for each change: response time, throughput, error rate and Apdex in the period before and after it. That turns the classic question *"did the release do this?"* into a two-second check.

Record changes automatically from the pipeline, not by hand:

```
# New Relic CLI (version 0.104.5 or later)
newrelic changeTracking create \
  --entitySearch "name = 'easyTravel-backend' AND type = 'APPLICATION'" \
  --category Deployment --type Basic \
  --version "2.1.0" --commit "9f3c2ab" \
  --description "Booking service: loyalty points lookup" \
  --user "jenkins" \
  --changelog "https://git.example.com/easytravel/releases/2.1.0"
```

```
# The same through NerdGraph (deployment marker on one entity)
mutation {
  changeTrackingCreateDeployment(deployment: {
    entityGuid: "<entity GUID of easyTravel-backend>"
    version: "2.1.0"
    commit: "9f3c2ab"
    description: "Booking service: loyalty points lookup"
    user: "jenkins"
  }) { deploymentId entityGuid timestamp }
}
```

CI/CD integrations (GitHub Actions, Jenkins, GitLab and others) wrap the same API.

Correlating a regression with a release:

1. On the Summary, does the jump in response time or errors start at (or a little after) a marker? Gradual rollouts and delayed effects — caches, background jobs, increased traffic — can shift the start by minutes.
2. Compare before and after with the marker's impact summary, or with NRQL `COMPARE WITH`.

3. Open traces from **after** the marker and compare them with traces from **before** it. The new or changed segment is the regression.
4. Check logs in context for new warnings or exceptions after the marker.
5. Decide: roll back, toggle the feature off, or fix forward — and record that as a change too.

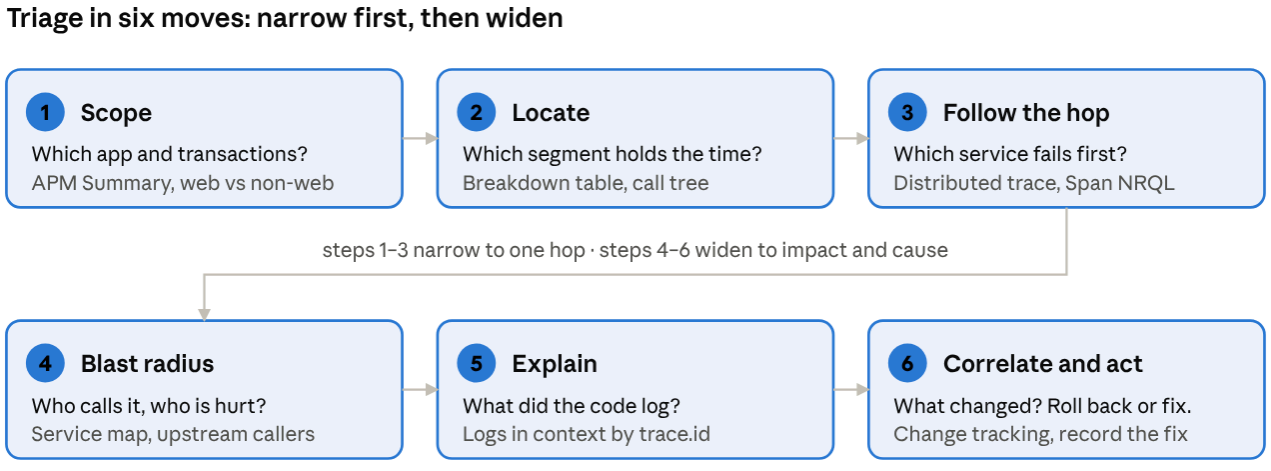
```
-- Response time of the booking call, this hour vs the same hour yesterday
SELECT average(duration) FROM Transaction
WHERE appName = 'easyTravel-backend' AND request.uri LIKE '%Booking%'
SINCE 1 hour ago COMPARE WITH 1 day ago TIMESERIES
```

> **Do Lab 1.3 — Correlate a post-deployment latency spike** in the Day 1 Lab Guide (50 min): record a deployment of easyTravel-backend 2.1.0 with the New Relic CLI, switch on `DBSpammingAuthWithAppDeployment` to simulate a bad release, then use the deployment marker, the service map, before/after traces and logs in context to prove the release caused the spike, and record the rollback.

Incident triage runbook and NRQL cheat sheet

This is the whole of Day 1 on one page: the order to work in when an APM alert fires, and the queries that answer each question. Keep it open during the labs and during your next real incident.

The triage flow



The Day 1 triage flow · each step maps to a module and a lab

#	Step	Where in New Relic	You are done when…	Module / lab
1	**Scope**	APM Summary: web and non-web charts, Apdex, error rate, throughput; *Transactions* sorted by *Most time consuming*	You can name the app, the transaction(s) and the start time	M1 · Lab 1.1
2	**Locate**	Transaction breakdown table; a slow transaction trace (call tree, *Identical Calls*, slow segments)	You know the segment type: database, external, or code — and whether it is one slow call or many fast ones (N+1)	M1 · Lab 1.1
3	**Follow the hop**	Distributed tracing: filter on errors or duration, open the waterfall, read the error span's attributes	You can name the first failing or slow span, its service and its `error.class`	M2 · Lab 1.2
4	**Blast radius**	Service map / dependencies of that service; error rate of each caller	You can list affected services and user journeys, and set the severity	M3 · Lab 1.3
5	**Explain**	The span's or error's **Logs** tab; Logs filtered by `trace.id`	You have the log line(s) that explain the failure	M3 · Lab 1.3
6	**Correlate and act**	Deployment markers and change tracking impact; before/after traces	You know whether a change caused it, you rolled back or fixed it, and you **recorded that action as a change**	M3 · Lab 1.3

Three rules that save the most time:

- **Symptom vs cause.** The service that alerts is rarely the one that is broken. Follow the time (or the error) downstream until it stops moving.
- **Count before you time.** A segment with a high *call count* and a low *average* is an N+1 pattern; making each call faster will not fix it.
- **Write it down.** Post the trace link, the query and the marker in the incident channel as you go — it becomes the timeline for the post-incident review.

NRQL cheat sheet

Replace `easyTravel-backend` and the transaction names with your own. All queries work in the query builder (*Query your data*) and as dashboard widgets.

Scope — transactions (Module 1)

```
-- Slowest transactions in the last 30 minutes, web and non-web
SELECT average(duration), percentile(duration, 95), count(*) FROM Transaction
WHERE appName = 'easyTravel-backend' FACET transactionType, name
SINCE 30 minutes ago LIMIT 20

-- Where the time goes: database vs external vs everything else
SELECT average(duration) AS 'total', average(databaseDuration) AS 'db',
       average(externalDuration) AS 'external' FROM Transaction
WHERE appName = 'easyTravel-backend' FACET name SINCE 30 minutes ago

-- N+1 hunting: database calls per transaction
SELECT average(databaseCallCount), max(databaseCallCount) FROM Transaction
WHERE appName = 'easyTravel-backend' FACET name SINCE 30 minutes ago

-- Error rate per transaction
SELECT percentage(count(*), WHERE error IS true) AS 'error %' FROM
Transaction
WHERE appName = 'easyTravel-backend' FACET name SINCE 30 minutes ago
```

Follow the hop — spans (Module 2)

```
-- Slowest downstream calls (database and external spans)
SELECT average(duration.ms), percentile(duration.ms, 95), count(*) FROM Span
WHERE appName = 'easyTravel-backend' AND span.kind = 'client'
FACET name SINCE 30 minutes ago

-- Failing spans by service, operation and error
SELECT count(*) FROM Span WHERE error.class IS NOT NULL
FACET appName, name, error.class SINCE 30 minutes ago

-- All spans of one trace, in order (paste a trace.id)
SELECT timestamp, appName, name, duration.ms, error.class, parentId, id FROM
Span
WHERE trace.id = '<trace id>' ORDER BY timestamp LIMIT 100

-- Broken traces: entry spans that should have had a parent
SELECT count(*) FROM Span WHERE appName = 'easyTravel-backend'
AND nr.entryPoint IS true AND parentId IS NULL FACET name SINCE 1 hour ago
```

Impact and context (Module 3)

```
-- Blast radius: callers and entry points with errors
SELECT uniqueCount(trace.id) FROM Span WHERE error.class IS NOT NULL
FACET appName, name SINCE 15 minutes ago

-- Log lines for one request
SELECT timestamp, entity.name, level, message FROM Log
WHERE trace.id = '<trace id>' ORDER BY timestamp

-- New error messages since the deploy
SELECT count(*) FROM Log WHERE entity.name = 'easyTravel-backend' AND level =
'ERROR'
FACET message SINCE 30 minutes ago COMPARE WITH 1 hour ago

-- Before vs after a release (TIMESERIES shows the step)
SELECT average(duration), percentage(count(*), WHERE error IS true) FROM
Transaction
WHERE appName = 'easyTravel-backend' SINCE 1 hour ago COMPARE WITH 1 day ago
TIMESERIES

-- Recorded deployments for the app
SELECT timestamp, version, description, user, changelog FROM Deployment
WHERE appName = 'easyTravel-backend' SINCE 1 week ago
```

> **Tip.** Event and attribute names can vary slightly between agent versions and account settings. If a query returns nothing, run `SELECT keyset() FROM Span SINCE 30 minutes ago` (or `FROM Transaction`, `FROM Log`) to see the attributes your data actually has.

Knowledge check, answers and sources

Try these after the labs, without the guide open. Answers follow the questions.

Questions

1. During an incident the *Web transactions time* chart is flat but users report that booking confirmations arrive late. Where should you look next, and why?
2. A transaction trace shows a MongoDB segment with **call count 240** and **average 1.8 ms**. What is the likely problem, and what fix does it point to?
3. In the trace details call tree, what do **Identical Calls** and the grouped *fast calls* tell you?
4. Why can a transaction named `/services/*` (or `WebTransaction/Servlet/default`) hide a real problem, and what are two ways to fix it?
5. What does a key transaction give you that a normal transaction does not?

6. Name the two HTTP headers defined by W3C Trace Context, and say which one carries the trace ID.
7. The frontend span in a trace shows an external call to the backend, but there is no backend span under it. Give three possible causes.
8. With head-based sampling, why can the trace of one particular failed request be missing, and what does Infinite Tracing change?
9. Write an NRQL query that lists failing spans by service, operation and error class for the last 30 minutes.
10. On the service map the frontend is red and the backend is red. How do you decide which one is the cause?
11. What must be true for a span's **Logs** tab to show the log lines of that request?
12. Response time rose at 14:05. A deployment marker shows 14:02. What three checks confirm (or rule out) the release as the cause?

Answers

1. **Non-web (background) transactions.** Confirmation work runs outside the HTTP request (jobs, consumers, schedulers) and is charted separately on the Summary. Switch the chart to non-web, then sort *Transactions* by type.
2. **An N+1 pattern:** each call is fast, but the code calls the database once per item in a loop. Fix it by batching the query (one call for the whole list), adding a join or caching — not by tuning the single query.
3. *Identical Calls* flags four or more consecutive identical calls — a strong N+1 hint. *Fast calls* groups many calls under 7.5 ms each: individually cheap, together they can be most of the time.
4. Many different operations share one name, so a slow or failing operation is averaged away among fast ones. Fix it with framework-aware naming (an instrumented framework, or agent naming settings) or with the agent API (`NewRelic.setTransactionName`) — while keeping names low-cardinality (no IDs in names).
5. Its own Apdex T threshold (web transactions), its own page and alert conditions, and visibility across the account — for the few business-critical operations you watch first.
6. `traceparent` (version, **trace ID**, parent span ID, flags) and `tracestate` (vendor-specific data, such as New Relic's entry).
7. Any three of: the backend agent is not running or not connected; distributed tracing is disabled on one side; a proxy, gateway or load balancer strips the headers; the call goes through a library the agent does not instrument (no header injection); asynchronous hand-off loses context; the backend reports to a different account without trust set up; the backend span was dropped by sampling or limits.

8. Head-based sampling decides at the **start** of the trace, before anyone knows it will fail, so most traces are not kept (the agent targets about 10 sampled transactions per minute by default). Infinite Tracing sends **100%** of spans to a trace observer, which keeps traces **after** they finish — including all error traces and duration outliers.
9. `SELECT count(*) FROM Span WHERE error.class IS NOT NULL FACET appName, name, error.class SINCE 30 minutes ago`
10. Follow the dependency direction: the frontend calls the backend, so check whether the frontend's errors are its calls to the backend (external errors) and whether the backend's errors are its **own** (datastore, code). The cause is the **deepest** service whose own errors or latency rose first.
11. The application must log through a supported framework, the agent must have log forwarding (or local decoration plus a log shipper) enabled, and the log line must be written on the request's thread while the trace is active, so it carries the same `trace.id` and `span.id`.
12. (a) The change in the charts starts at or shortly after the marker — and not before it; (b) traces after the marker show a new or slower segment that traces before it do not have; (c) logs in context after the marker show new warnings or errors on that code path. Optionally confirm with `COMPARE WITH` and, finally, with the rollback restoring the old numbers.

Sources

- New Relic docs — Introduction to transaction traces and Transaction trace details
- New Relic docs — How New Relic distributed tracing works
- New Relic docs — Introduction to Infinite Tracing
- New Relic docs — Service maps
- New Relic docs — Java agent: configure logs in context and logs in context settings
- New Relic docs — Change tracking introduction and Change tracking with NerdGraph
- W3C — Trace Context recommendation
- Dynatrace — easyTravel-Docker on GitHub (the lab application)

New Relic's UI changes often; screen names and menu paths in this guide were checked against the documentation above at the time of writing and may differ slightly in your account.